Dog Integration for Cursor — Individual-Frame Sprites

RoboPet · Living Pet · Drop-in code to make the robot dog walk around the living room. PixelLab exports individual frame PNGs, so this version loads frame lists.

1. Why this version (individual frames)

PixelLab downloads each animation as separate frame PNGs (8 frames = 8 files) with no spritesheet option, so we load each animation as a list of single-frame sprites. Your download format is exactly right now.

2. Asset folder layout

Create one subfolder per animation under assets/images/dog/ and rename that animation frames to 0.png, 1.png, 2.png ... in playback order. Only fill the folders you have; missing animations are skipped automatically.

```
assets/images/dog/
    idle/        0.png 1.png 2.png ...
    walk/        0.png 1.png ...
    happy/       0.png ...
    eat/         0.png ...
    sleep/       0.png ...
    low_power/   0.png ...
    attack/      0.png ...
```

3. pubspec.yaml

Add the folders under flutter > assets, then run flutter pub get.

```
flutter:
  assets:
    - assets/images/dog/idle/
    - assets/images/dog/walk/
    - assets/images/dog/happy/
    - assets/images/dog/eat/
    - assets/images/dog/sleep/
    - assets/images/dog/low_power/
    - assets/images/dog/attack/
```

4. lib/game/components/robot_dog.dart

```
import 'dart:math';
import 'package:flame/components.dart';
import 'package:flame/effects.dart';
import 'package:flutter/material.dart' show Colors;

/// Animation states for the robot dog.
```

```
enum DogAnim { idle, walk, happy, eat, sleep, lowPower, attack }

/// One animation: folder name + frame count + seconds per frame + loop.
class _AnimSpec {
  final String folder;
  final int frames;
  final double step;
  final bool loop;
  const _AnimSpec(this.folder, this.frames, this.step, {this.loop = true});
}

class RobotDog extends SpriteAnimationGroupComponent<DogAnim> {
  RobotDog({required Vector2 position})
      : super(
          position: position,
          size: Vector2.all(80),
          anchor: Anchor.bottomCenter,
          priority: 10,
        );

  // ---- Set frame counts to match YOUR exported PNGs (default 8). ----
  static const Map<DogAnim, _AnimSpec> _config = {
    DogAnim.idle:     _AnimSpec('idle', 8, 0.12),
    DogAnim.walk:     _AnimSpec('walk', 8, 0.08),
    DogAnim.happy:    _AnimSpec('happy', 8, 0.09),
    DogAnim.eat:      _AnimSpec('eat', 8, 0.10),
    DogAnim.sleep:    _AnimSpec('sleep', 8, 0.22),
    DogAnim.lowPower: _AnimSpec('low_power', 8, 0.14),
    DogAnim.attack:   _AnimSpec('attack', 8, 0.07, loop: false),
  };

  // Wander tuning (matches the old placeholder cube).
  static const double _speed = 55.0;
  static const double _pauseMin = 1.2;
  static const double _pauseMax = 3.0;
  static const double _wanderRange = 130.0;

  final _rng = Random();
  Vector2 _target = Vector2.zero();
  double _pauseTimer = 0;
  bool _pausing = true;
  bool _wandering = true;
  bool _facingRight = true;

  @override
  Future<void> onLoad() async {
    final Map<DogAnim, SpriteAnimation> loaded = {};
    for (final entry in _config.entries) {
      try {
        final frames = <Sprite>[];
        for (var i = 0; i < entry.value.frames; i++) {
          frames.add(await Sprite.load('dog/${entry.value.folder}/$i.png'));
        }
        loaded[entry.key] = SpriteAnimation.spriteList(
          frames,
          stepTime: entry.value.step,
          loop: entry.value.loop,
        );
      } catch (_) {
        // This animation has no PNGs yet -> skip it gracefully.
      }
    }
```

```
  animations = loaded;
  current = loaded.containsKey(DogAnim.idle)
      ? DogAnim.idle
      : (loaded.isNotEmpty ? loaded.keys.first : null);

  _target = position.clone();
  _schedulePause();
}

@override
void update(double dt) {
  super.update(dt);
  if (!_wandering) return;

  if (_pausing) {
    _pauseTimer -= dt;
    if (_pauseTimer <= 0) {
      _pausing = false;
      _target = Vector2(
        (_rng.nextDouble() * 2 - 1) * _wanderRange,
        position.y,
      );
      _faceTowards(_target.x);
      _setAnim(DogAnim.walk);
    }
  } else {
    final dir = _target - position;
    if (dir.length < 2) {
      position.setFrom(_target);
      _pausing = true;
      _schedulePause();
      _setAnim(DogAnim.idle);
    } else {
      position += dir.normalized() * _speed * dt;
    }
  }
}

void _schedulePause() =>
    _pauseTimer = _pauseMin + _rng.nextDouble() * (_pauseMax - _pauseMin);

void _faceTowards(double targetX) {
  final goRight = targetX >= position.x;
  if (goRight != _facingRight) {
    _facingRight = goRight;
    flipHorizontallyAroundCenter();
  }
}

void _setAnim(DogAnim a) {
  final map = animations;
  if (map != null && map.containsKey(a) && current != a) {
    current = a;
  }
}

// ---- Optional hooks for later care logic ----

void startWander() {
  _wandering = true;
  _pausing = true;
  _schedulePause();
```

```
  }

  void stopWander() => _wandering = false;

  void applyCareState({required double battery, required bool asleep}) {
    if (asleep) {
      stopWander();
      _setAnim(DogAnim.sleep);
    } else if (battery < 0.20) {
      stopWander();
      _setAnim(DogAnim.lowPower);
    } else {
      startWander();
    }
  }

  // Code-based hurt: white flash + shake (no hurt sprite needed).
  void hit() {
    add(ColorEffect(
      Colors.white,
      EffectController(duration: 0.08, alternate: true),
      opacityTo: 0.85,
    ));
    add(MoveEffect.by(
      Vector2(_facingRight ? -8 : 8, 0),
      NoiseEffectController(duration: 0.3, frequency: 24),
    ));
  }
}
```

5. lib/game/rooms/living_room.dart (drop-in, keeps your coordinates)

Adjust the import path if your folders differ.

```
import 'package:flame/components.dart';
import 'package:flame/game.dart';
import 'package:flutter/material.dart' show Color, Paint;

import '../components/robot_dog.dart';

class LivingRoomWorld extends World {
  @override
  Future<void> onLoad() async {
    await super.onLoad();

    // Dark background (swap for a room SpriteComponent later).
    add(RectangleComponent(
      size: Vector2(400, 700),
      position: Vector2(-200, -350),
      paint: Paint()..color = const Color(0xFF0D0F1A),
    ));

    // Floor line.
    add(RectangleComponent(
      size: Vector2(400, 12),
      position: Vector2(-200, 120),
      paint: Paint()..color = const Color(0xFF1A2040),
    ));

    // Robot dog (replaces the old placeholder cube; same position).
    add(RobotDog(position: Vector2(0, 90)));
```

```
  }
}
```

6. Set your real frame counts

In robot_dog.dart the _config map lists each animation with a frame count (default 8). Change each number to match how many PNGs you put in that folder (mostly 8, some 6 or 9). The dog size is 80 and sits on the floor at y = 90, matching your old placeholder.

7. Run + Cursor task

Paste this to Cursor and attach this PDF:

```
Implement a wandering animated robot dog in my Flutter + Flame game.
1. Create lib/game/components/robot_dog.dart exactly as in section 4.
2. Replace lib/game/rooms/living_room.dart with section 5 (keep my coordinates).
3. Add the dog asset folders to pubspec.yaml as in section 3, then run flutter pub get.
4. I will drop frame PNGs named 0.png, 1.png, ... into each assets/images/dog/<anim>/
folder. Set each animation frame count in _config to match the number of files. Leave
folders I have not filled; the code ignores missing animations.
5. Run the app. The dog should idle, then walk back and forth along the floor and flip
to face its direction.
```

8. Later: real background

When your room PNG is ready, replace the dark background RectangleComponent in living_room.dart with a SpriteComponent that loads rooms/living_room.png sized to the world. The dog and wander code stay the same.